PROYECTO 2

APLICACIÓN REST CLIENTE-SERVIDOR

Luis Cervelló Mediel

ÍNDICE

1. Explicación del proyecto (Página 2)
2. Tecnologías utilizadas (Página 3)
3. Funcionamiento del sistema (Página 4-5)
4. La base de datos (Página 6)
5. El servidor (API REST) (Página 9-14)
6. El cliente (aplicación de consola) (Página 15-18)
7. Pruebas del sistema (Página 19-20)
8. Cómo ejecutar el proyecto (Página 21-24)
9. Repositorio GitHub (Página 25-26)
10. Dificultades encontradas (Página 27)

1. Explicación del proyecto

He creado una aplicación para gestionar una tienda. El sistema permite:

- Crear categorías (Electrónica, Ropa, Alimentación...)
- Crear productos dentro de esas categorías
- Ver, modificar y borrar tanto categorías como productos

El proyecto tiene dos partes principales:

SERVIDOR: Es un programa que está siempre encendido esperando peticiones. Guarda los datos en MySQL y responde cuando el cliente le pide información. CLIENTE: Es la aplicación que usa el usuario. Muestra menús por consola y se comunica con el servidor para hacer las operaciones.

Programación de Servicios y Procesos: La comunicación entre cliente y servidor usando HTTP (peticiones GET, POST, PUT, DELETE).

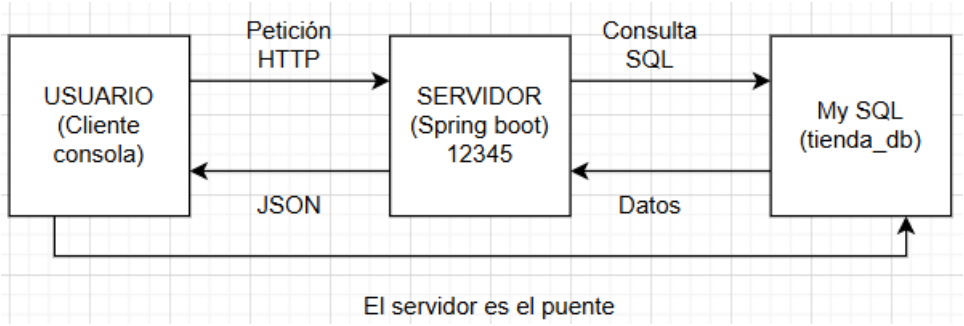
Acceso a Datos: Guardar y leer datos de MySQL usando JPA (la forma fácil de usar bases de datos en Java).

Desarrollo de Interfaces: Los menús de la aplicación de consola que ve el usuario.

2. Tecnologías utilizadas

Tecnología	¿Para qué sirve?
Java	El lenguaje de programación. Todo el código está escrito en Java.
Spring Boot	Un framework para crear servidores web en Java.
MySQL	La base de datos donde se guardan las categorías y productos.
Maven	Descarga automáticamente las librerías que necesito y compila el proyecto.
JPA / Hibernate	Permite guardar objetos Java directamente en la base de datos sin escribir SQL a mano.
GSON	Convierte objetos Java a JSON y viceversa (JSON es el formato que usa la API).
JUnit 5	Para escribir y ejecutar pruebas automáticas del código.
Swagger	Crea una página web para probar la API desde el navegador.

3. Funcionamiento del sistema



Ejemplo de crear un producto:

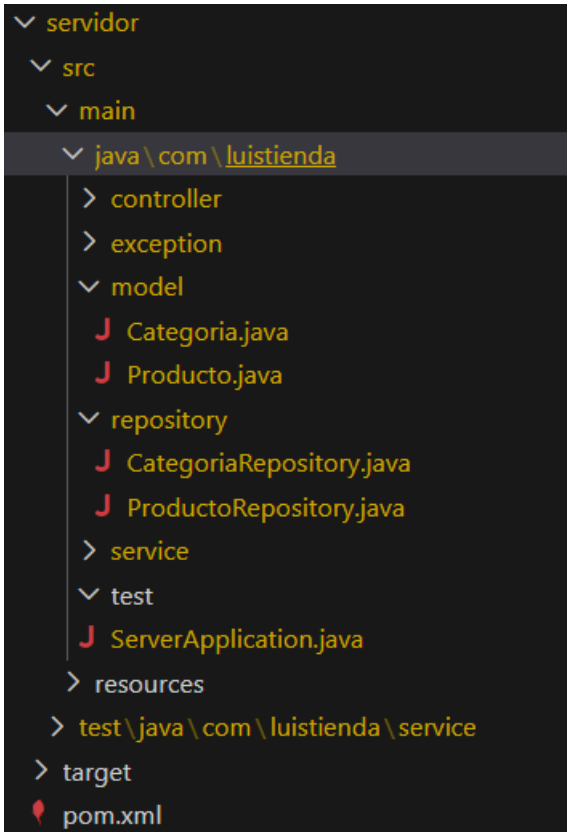
Cuando el usuario quiere crear un producto nuevo, esto es lo que pasa:

1. El usuario selecciona "Crear producto" en el menú del cliente
2. El usuario escribe el nombre, precio, stock y categoría
3. El cliente crea un objeto Producto con esos datos
4. El cliente convierte el objeto a JSON (texto)
5. El cliente envía una petición POST al servidor con ese JSON
6. El servidor recibe la petición en el Controller
7. El Controller llama al Service para procesar los datos
8. El Service valida que todo esté bien y llama al Repository
9. El Repository guarda el producto en MySQL
10. El servidor devuelve el producto creado (con su ID nuevo) al cliente
11. El cliente muestra por pantalla: "Producto creado con éxito"

Las capas del servidor:

El servidor está organizado en capas. Cada capa tiene una responsabilidad:

Capa	¿Qué hace?	Ejemplo
Controller	Recibe las peticiones HTTP y devuelve las respuestas	CategoriaController.java
Service	Contiene la lógica de negocio (validaciones, reglas...)	CategoriaService.java
Repository	Habla con la base de datos (guardar, leer, borrar...)	CategoriaRepository.java
Model	Son las clases que representan los datos (entidades)	Categoria.java, Producto.java



4. La base de datos

Las tablas:

La base de datos se llama tienda_db y tiene 2 tablas:

Tabla: categorias

Campo	Tipo	¿Para qué sirve?
id	BIGINT (número grande)	Identificador único. Se genera automáticamente (1, 2, 3...)
nombre	VARCHAR(100)	El nombre de la categoría (ej: "Electrónica")
descripcion	VARCHAR(255)	Una descripción opcional

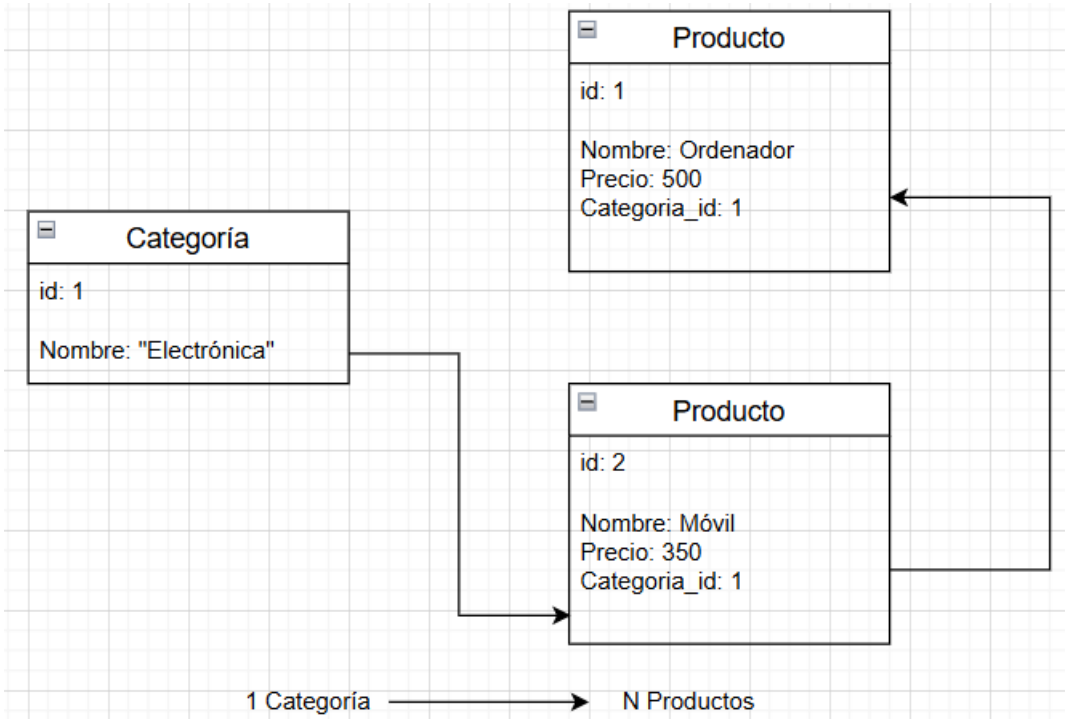
Tabla: productos

Campo	Tipo	¿Para qué sirve?
id	BIGINT	Identificador único del producto
nombre	VARCHAR(100)	El nombre del producto (ej: "Laptop Dell")
precio	DOUBLE	El precio en euros (ej: 599.99)
stock	INT	Cuántas unidades hay disponibles
categoria_id	BIGINT	A qué categoría pertenece (relación con la otra tabla)

Relación entre tablas:

Una categoría puede tener muchos productos. Por ejemplo, la categoría "Electrónica" puede tener: Ordenador, Móvil, Tablet, etc.

Esto se llama relación 1 a N (uno a muchos):



id	descripcion	nombre
1	Dispositivos y aparatos electrónicos	Electrónica
2	Prendas de vestir	Ropa
3	Productos alimenticios	Alimentación
NULL	NULL	NULL

id	nombre	precio	stock	categoria_id
1	Ordenador Portátil	899.99	15	1
2	Smartphone Samsung	549	30	1
3	Auriculares Bluetooth	45.99	50	1
4	Camiseta Básica	19.99	100	2
5	Pantalón Vaquero	39.95	40	2
6	Arroz Integral 1kg	2.5	200	3
NULL	NULL	NULL	NULL	NULL

Configuración:

La conexión a MySQL se configura en el archivo application.properties:

Projecto_ClienteServidor > servidor > src > main > resources > application.properties

```
# Configuración del servidor
server.port=12345

# Configuración de la base de datos MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/tienda_db?useSSL=false&se
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Configuración de JPA/Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

# Configuración de Swagger
springdoc.api-docs.path=/api-docs
springdoc.swagger-ui.path=/swagger-ui.html

# Logging
logging.level.org.springframework.web=INFO
logging.level.com.luistienda=DEBUG
```

Cuando arrancas el servidor, Hibernate (JPA) mira tus clases Java (Categoria.java, Producto.java) y crea las tablas automáticamente en MySQL si no existen. Si ya existen, las actualiza si hace falta.

Crear la base de datos:

En MySQL creo la base de datos tienda_db, las tablas se crean solas cuando arrancas el servidor.

5. El servidor API REST

Una API REST es una forma de que dos programas se comuniquen por Internet usando peticiones HTTP (como un navegador web, pero sin interfaz gráfica).

Método HTTP	¿Para qué se usa?	Ejemplo
GET	Obtener datos (leer)	GET /api/productos → devuelve todos los productos
POST	Crear algo nuevo	POST /api/productos → crea un producto nuevo
PUT	Modificar algo existente	PUT /api/productos/5 → modifica el producto con id=5
DELETE	Borrar algo	DELETE /api/productos/5 → borra el producto con id=5

Endpoints del servidor:

Un endpoint es una "dirección" del servidor que hace algo concreto. Mi servidor tiene 10 endpoints:

Endpoints de Categorías

Método	URL	¿Qué hace?
GET	/api/categorias	Devuelve todas las categorías
GET	/api/categorias/{id}	Devuelve una categoría por su ID
POST	/api/categorias	Crea una categoría nueva
PUT	/api/categorias/{id}	Modifica una categoría
DELETE	/api/categorias/{id}	Borra una categoría

Endpoints de Productos

Método	URL	¿Qué hace?
GET	/api/productos	Devuelve todos los productos
GET	/api/productos/{id}	Devuelve un producto por su ID
GET	/api/productos/categoria/{id}	Devuelve los productos de una categoría
POST	/api/productos	Crea un producto nuevo
PUT	/api/productos/{id}	Modifica un producto
DELETE	/api/productos/{id}	Borra un producto

Código del Controller

El CategoriaController recibe las peticiones HTTP.

```java
import com.luistienda.exception.ResourceNotFoundException;
import com.luistienda.model.Categoria;
import com.luistienda.service.CategoriaService;

/**
 * Controlador REST para gestionar Categorías.
 * Endpoints: /api/categorias
 */
@RestController
@RequestMapping("/api/categorias")
@CrossOrigin(origins = "*")
public class CategoriaController {

    @Autowired
    private CategoriaService categoriaService;

    // GET /api/categorias - Obtener todas
    @GetMapping
    public List<Categoria> obtenerTodas() {
        return categoriaService.obtenerTodas();
    }

    // GET /api/categorias/{id} - Obtener por ID
    @GetMapping("/{id}")
    public ResponseEntity<Categoria> obtenerPorId(@PathVariable Long id) {
        try {
            Categoria categoria = categoriaService.obtenerPorId(id);
            return ResponseEntity.ok(categoria);
        } catch (ResourceNotFoundException e) {
            return ResponseEntity.notFound().build();
        }
    }

    // POST /api/categorias - Crear nueva
    @PostMapping
    public ResponseEntity<Categoria> crear(@RequestBody Categoria categoria)
```

Código del Service

El CategoriaService contiene la lógica de negocio:

```java
import com.luistienda.exception.ResourceNotFoundException;
import com.luistienda.model.Categoria;
import com.luistienda.repository.CategoriaRepository;

/**
 * Servicio con la lógica de negocio para Categorías.
 */
@Service
public class CategoriaService {

    @Autowired
    private CategoriaRepository categoriaRepository;

    // Obtener todas las categorías
    public List<Categoria> obtenerTodas() {
        return categoriaRepository.findAll();
    }

    // Obtener una categoría por ID
    public Categoria obtenerPorId(Long id) {
        return categoriaRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Categoría no
    }

    // Crear una nueva categoría
    public Categoria crear(Categoria categoria) {
        if (categoria.getNombre() == null || categoria.getNombre().isEmpty()) {
            throw new IllegalArgumentException(s: "El nombre es obligatorio");
        }
        return categoriaRepository.save(categoria);
    }

    // Actualizar una categoría
    public Categoria actualizar(Long id, Categoria datos) {
        Categoria categoria = obtenerPorId(id);
```

Código del Repository

El Repository es muy simple porque Spring Data JPA hace casi todo. JpaRepository ya incluye los métodos básicos (CRUD). Solo tenemos que decirle qué entidad usa (Categoria) y qué tipo tiene el ID (Long). Spring hace el resto.

```java
/**
 * Repositorio para acceder a datos de Categoria.
 * Spring Data JPA implementa automáticamente los métodos CRUD.
 */
@Repository
public interface CategoriaRepository extends JpaRepository<Categoria, Long>

    // Método personalizado para buscar por nombre
    Optional<Categoria> findByNombre(String nombre);
}
```

Código de la Entidad

La entidad Categoria representa la tabla en la base de datos

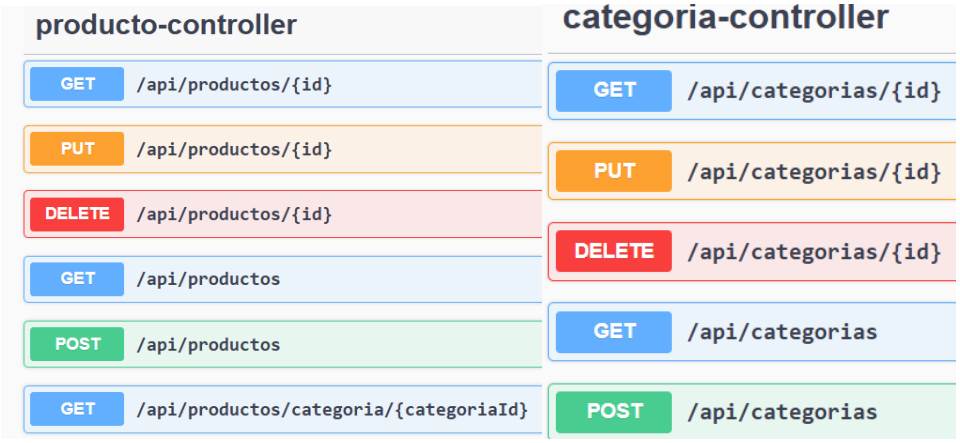
```java
/**
 * Entidad que representa una categoría de productos.
 * Se mapea a la tabla "categorias" en la base de datos.
 */
@Entity
@Table(name = "categorias")
public class Categoria {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String nombre;

    @Column(length = 255)
    private String descripcion;

    // Constructor vacío (necesario para JPA)
    public Categoria() {
    }

    // Constructor con parámetros
    public Categoria(String nombre, String descripcion) {
        this.nombre = nombre;
        this.descripcion = descripcion;
```

Documentación con Swagger

He añadido Swagger al proyecto. Swagger crea una página web donde puedes ver y probar todos los endpoints de la API desde el navegador.

Para acceder a Swagger:

1. Arranca el servidor
2. Abre el navegador en: http://localhost:12345/swagger-ui.html
3. Verás todos los endpoints y podrás probarlos



Curl

```
curl -X 'GET' \
  'http://localhost:12345/api/categorias' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:12345/api/categorias
```

Server response

Code	Details
200	Response body

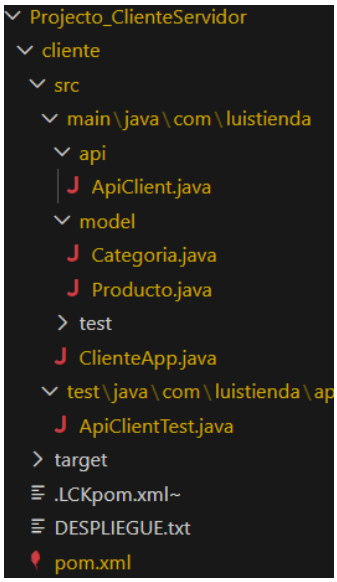
```
[
  {
    "id": 1,
    "nombre": "Electrónica",
    "descripcion": "Dispositivos y aparatos electrónicos"
  },
  {
    "id": 2,
    "nombre": "Ropa",
    "descripcion": "Prendas de vestir"
  },
  {
    "id": 3,
    "nombre": "Alimentación",
    "descripcion": "Productos alimenticios"
  }
]
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Sun,01 Mar 2026 22:04:29 GMT
keep-alive: timeout=60
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
```

6. El cliente (Aplicación de consola)

El cliente es una aplicación de consola que el usuario ejecuta para interactuar con el sistema. Muestra menús y permite hacer todas las operaciones CRUD.




```
(.venv) C:\Users\luisc\Documents\Java\Projecto_ClienteServidor\cliente>java -cp "targ
et\classes;target\cliente-tienda-1.0.0.jar" com.luistienda.ClienteApp
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
```

```
   CLIENTE TIENDA LUIS
Conectando a servidor en puerto 12345...

MENÚ PRINCIPAL-TIENDA LUIS
1. Gestionar Categorías
2. Gestionar Productos
0. Salir
Elige opción: 1

 CATEGORÍAS
1. Ver todas
2. Crear nueva
3. Buscar por ID
4. Actualizar
5. Eliminar
0. Volver
Elige opción: 
```

Código del ApiClient

El ApiClient es la clase que se comunica con el servidor.
Usa HttpURLConnection (incluido en Java, no necesita librerías externas):

```java
/**
 * Cliente HTTP para consumir la API REST del servidor.
 * Usa HttpURLConnection de Java estándar para las peticiones HTTP.
 */
public class ApiClient {

    // URL base del servidor
    private static final String BASE_URL = "http://localhost:12345/api";

    // Objeto Gson para convertir entre Java y JSON
    private static final Gson gson = new Gson();

    //  MÉTODOS PARA CATEGORÍAS

    /**
     * Obtiene todas las categorías del servidor.
     */
    public static List<Categoria> obtenerCategorias() throws Exception {
        String respuesta = hacerPeticionGET(BASE_URL + "/categorias");
        return gson.fromJson(respuesta, new TypeToken<List<Categoria>>()
    }
```

1. Crea una conexión HTTP a la URL del servidor
2. Configura el método (GET, POST, PUT, DELETE)
3. Si es POST/PUT, envía los datos en formato JSON
4. Lee la respuesta del servidor
5. Convierte el JSON de vuelta a objetos Java con GSON

Crear una categoría

Esto es lo que ve el usuario cuando crea una categoría

```
 CATEGORÍAS
1. Ver todas
2. Crear nueva
3. Buscar por ID
4. Actualizar
5. Eliminar
0. Volver
Elige opción: 2

>> CREAR CATEGORÍA
Nombre: Deportes
Descripción: Raqueta de tenis
Categoría creada con ID: 6
```

```
 CATEGORÍAS
1. Ver todas
2. Crear nueva
3. Buscar por ID
4. Actualizar
5. Eliminar
0. Volver
Elige opción: 1

>> LISTA DE CATEGORÍAS:
ID: 1 | Nombre: Electrónica | Descripción: Dispositivos y aparatos electrónic
os
ID: 2 | Nombre: Ropa | Descripción: Prendas de vestir
ID: 3 | Nombre: Alimentación | Descripción: Productos alimenticios
ID: 6 | Nombre: Deportes | Descripción: Raqueta de tenis
```

Crear un producto

```
 PRODUCTOS
1. Ver todos
2. Crear nuevo
3. Buscar por ID
4. Buscar por categoría
5. Actualizar
6. Eliminar
0. Volver
Elige opción: 2

>> CREAR PRODUCTO
Nombre: LG TV
Precio: 600
Stock: 8
ID de categoría: 1
Producto creado con ID: 7
```

```
Elige opción: 1

>> LISTA DE PRODUCTOS:
ID: 1 | Ordenador Portátil | Precio: 899.99€ | Stock: 15 | Categoría: El
nica
ID: 2 | Smartphone Samsung | Precio: 549.0€ | Stock: 30 | Categoría: Ele
ica
ID: 3 | Auriculares Bluetooth | Precio: 45.99€ | Stock: 50 | Categoría:
rónica
ID: 4 | Camiseta Básica | Precio: 19.99€ | Stock: 100 | Categoría: Ropa
ID: 5 | Pantalón Vaquero | Precio: 39.95€ | Stock: 40 | Categoría: Ropa
ID: 6 | Arroz Integral 1kg | Precio: 2.5€ | Stock: 200 | Categoría: Alim
ión
ID: 7 | LG TV | Precio: 600.0€ | Stock: 8 | Categoría: Electrónica
```

7. Pruebas del sistema

He creado pruebas automáticas usando JUnit 5. Las pruebas comprueban que todo funciona correctamente conectándose a la base de datos real.

En total hay 12 pruebas:

Archivo	Nº de pruebas	¿Qué prueba?
CategoriaServiceTest.java	5	Crear, leer, actualizar y borrar categorías
ProductoServiceTest.java	7	Crear, leer, actualizar, borrar productos y filtrar por categoría

Esta prueba comprueba que se puede crear una categoría correctamente:

```java
 * Tests de integracion para CategoriaService.
 * Conecta a la base de datos MySQL real.
 *
 * IMPORTANTE: MySQL debe estar activo para ejecutar estos tests.
 */
@SpringBootTest
public class CategoriaServiceTest {

    @Autowired
    private CategoriaService categoriaService;

    @Autowired
    private CategoriaRepository categoriaRepository;

    // Variable para guardar el ID de la categoria creada en los tests
    private Long categoriaTestId;

    /**
     * Limpia los datos de prueba despues de cada test.
     */
    @AfterEach
    void limpiarDatos() {
```

@SpringBootTest: Arranca toda la aplicación Spring Boot para que la prueba pueda usar los servicios y la base de datos real. Es una prueba de integración.

@AfterEach: Después de cada prueba, borra todos los datos que se han creado. Así cada prueba empieza con la base de datos limpia.

Ejecutar las pruebas

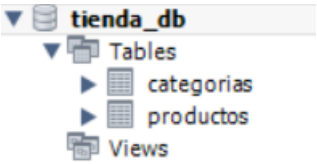
Desde servidor: mvn test

```
[INFO] --------------------------------[ jar ]---------------------------------
--
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ servidor-tienda --
-
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ servidor-tienda ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 10 source files with javac [debug release 17] to target\clas
ses
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ servidor-t
```

```
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.260
s -- in com.luistienda.service.ProductoServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
--
[INFO] Total time:  7.946 s
[INFO] Finished at: 2026-03-01T23:46:06+01:00
```

8. Cómo ejecutar el proyecto

En MySQL creamos la base de datos: tienda_bd.



Compilar los proyectos

Compilar el servidor

- cd servidor
- mvn clean package -DskipTests

Compilar el cliente

- cd cliente
- mvn clean package -DskipTests

Esto genera los archivos JAR.

cliente-tienda-1.0.0.jar

servidor-tienda-1.0.0.jar

Ejecutar el servidor

java -jar servidor/target/servidor-tienda-1.0.0.jar

```
java -jar target\servidor-tienda-1.0.0.jar
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=U
TF-8


  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.3.2)

2026-03-02T00:23:43.934+01:00  INFO 21032 --- [           main] com.luis
tienda.ServerApplication         : Starting ServerApplication v1.0.0 usi
ng Java 25 with PID 21032 (C:\Users\luisc\Documents\Java\Projecto Client
```

Ejecutar cliente

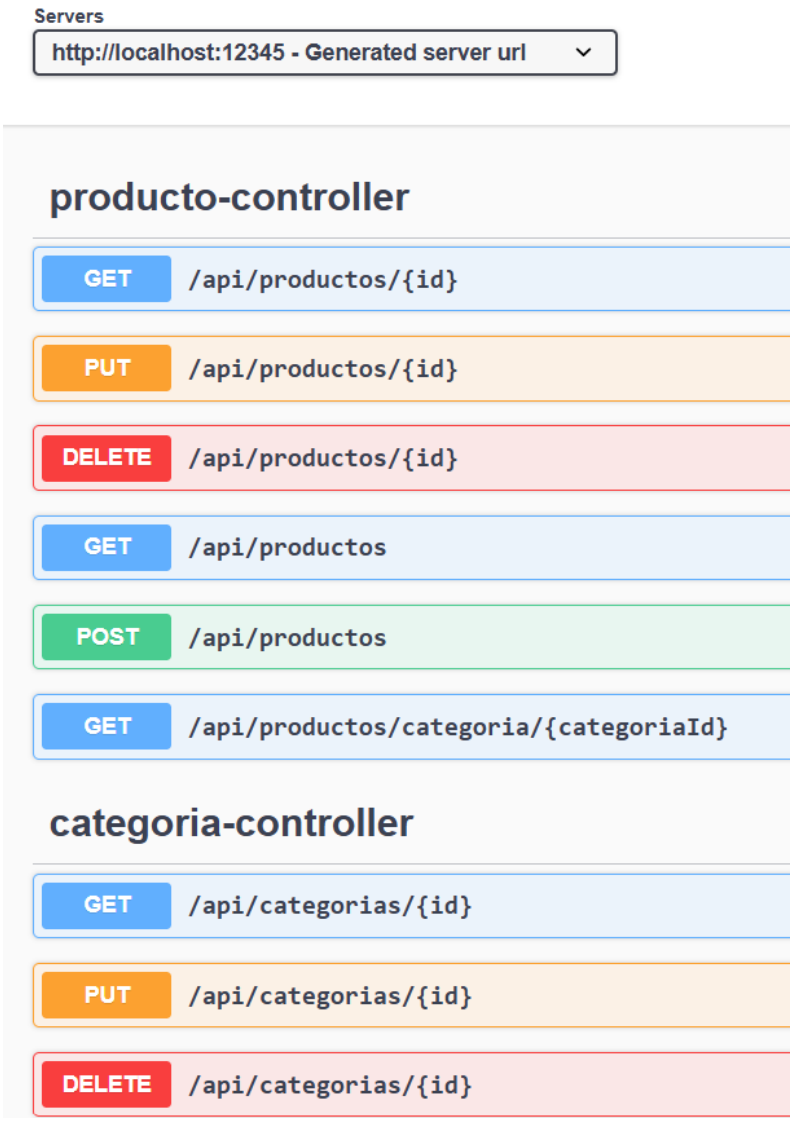
java -jar cliente/target/cliente-tienda-1.0.0-jar-with-dependencies.jar

```
(.venv) C:\Users\luisc\Documents\Java\Projecto_ClienteServidor\cliente>java -
cp "target\classes;target\cliente-tienda-1.0.0.jar" com.luistienda.ClienteApp
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

   CLIENTE TIENDA LUIS
Conectando a servidor en puerto 12345...

MENÚ PRINCIPAL-TIENDA LUIS
1. Gestionar Categorías
2. Gestionar Productos
0. Salir
Elige opción:
```

Acceder a la documentación

Con el servidor arrancado, puedes acceder a:

URL	¿Qué es?
http://localhost:12345/swagger-ui.html	Documentación interactiva de la API (Swagger)
servidor/target/site/apidocs/index.html	Documentación del código (Javadoc)



Cliente Tienda 1.0.0 API

Packages

Package
com.luistienda
com.luistienda.api
com.luistienda.model

Package com.luistienda

```
package com.luistienda
```

Related Packages

Package	Description
com.luistienda.api	
com.luistienda.model	

Classes

Class	Description
ClienteApp	Aplicación cliente de consola para gestionar la tienda.

Copyright © 2026. All rights reserved.

Classes

Class	Description
ApiClient	Cliente HTTP para consumir la API REST del servidor.

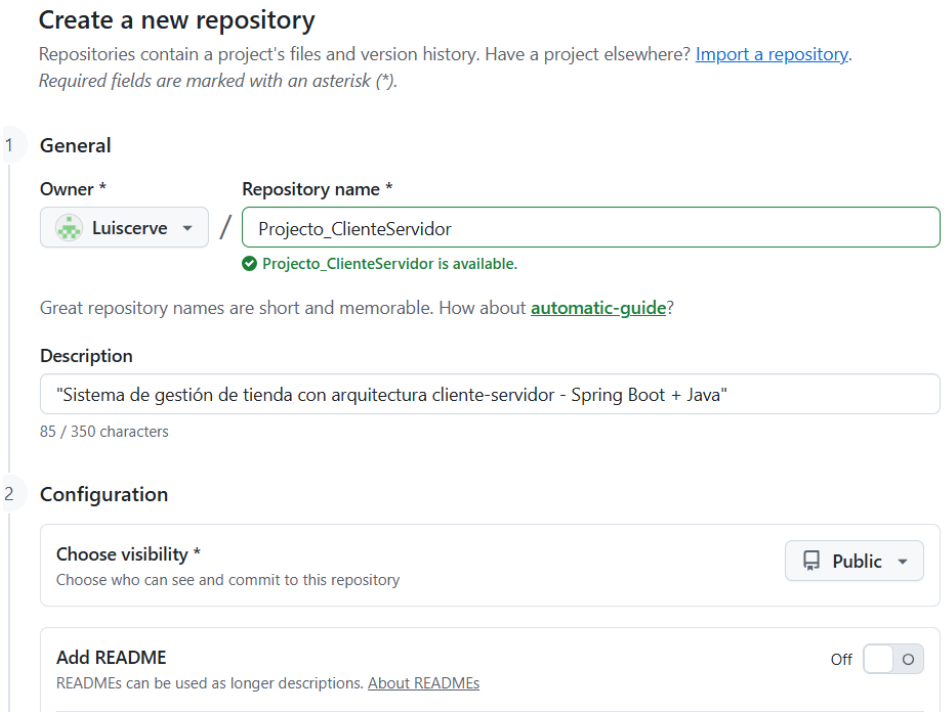
Copyright © 2026. All rights reserved.

Classes

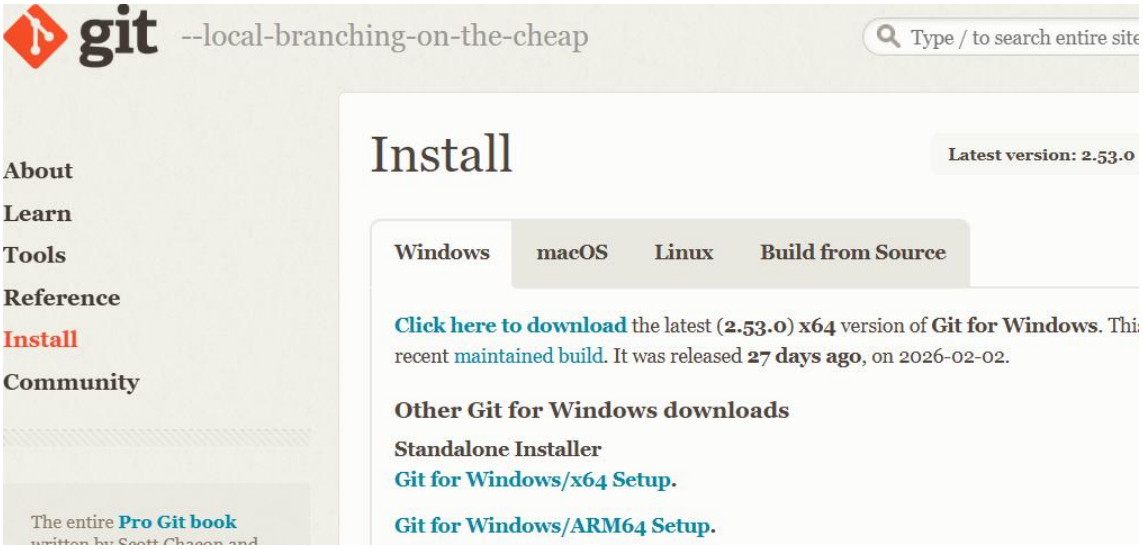
Class	Description
Categoria	Clase que representa una categoría de productos.
Producto	Clase que representa un producto de la tienda.

Copyright © 2026. All rights reserved.

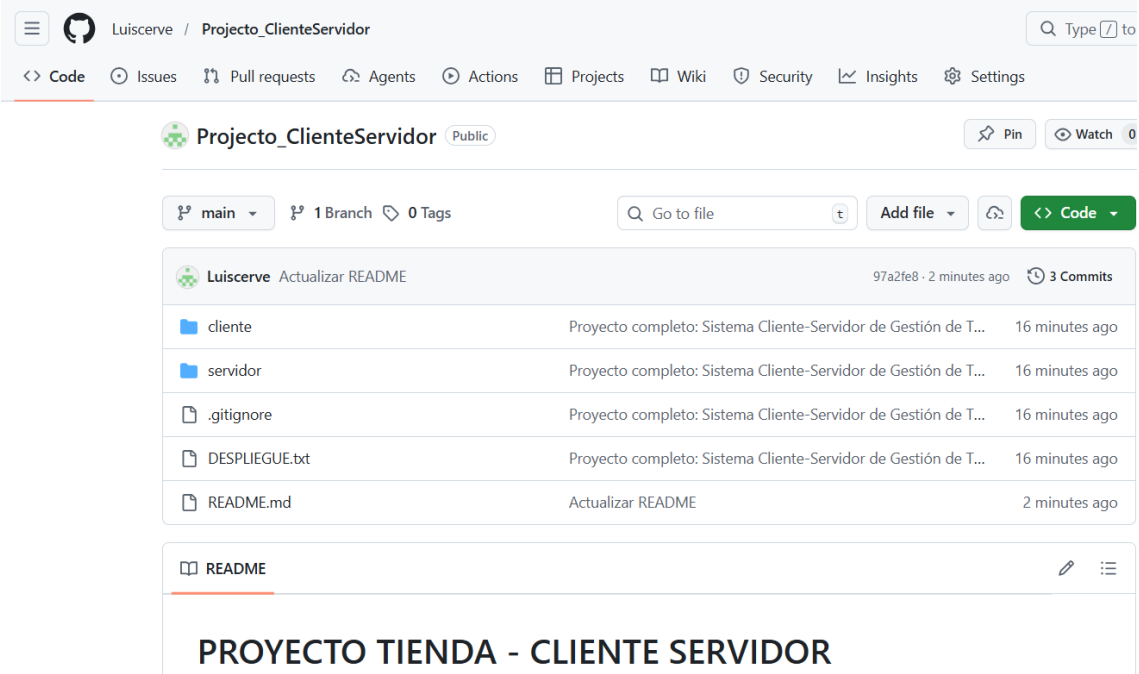
9. Repositorio GitHub



Instalar Git



```
C:\Users\luisc\Documents\Java\Projecto_ClienteServidor>git config --globa
l user.email "luiscervello43@gmail.com"

C:\Users\luisc\Documents\Java\Projecto_ClienteServidor>git remote add ori
gin https://github.com/Luiscerve/Projecto_ClienteServidor.git

C:\Users\luisc\Documents\Java\Projecto_ClienteServidor>git push -u origin
 main
Enumerating objects: 59, done.
Counting objects: 100% (59/59), done.
Delta compression using up to 12 threads
Compressing objects: 100% (44/44), done.
Writing objects: 100% (59/59), 33.10 KiB | 3.31 MiB/s, done.
Total 59 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/Luiscerve/Projecto_ClienteServidor.git
 * [new branch]      main -> main
```



10. Dificultades encontradas

Problema	Solución
Configurar la conexión a MySQL	Revisar application.properties y verificar que MySQL esté corriendo
Entender las anotaciones de Spring	Leer documentación y hacer pruebas paso a paso
Serializar/deserializar JSON	Usar GSON y definir bien las clases modelo
Las pruebas fallaban por datos residuales	Añadir @AfterEach para limpiar la base de datos
Instalación de Git	Instalar git y subir el repositorio